

TEST_IT_NOW_AND_TOMORROW

NetworkPeer — Test Everything Guide (Mac now, Linux tomorrow)

This guide walks you through **every feature the way you’ll show it to the boss**, right now on this Mac, then tomorrow from the locked-down Linux laptop (browser only — nothing to install there). It also covers the **cloud database** option and your **live AWS S3** account.

0. What changed in the frontend (verify these as you test)

#	Change	Where to see it
1	Stat-card labels + numbers are larger ("Jobs posted", "Jobs active", ...)	Client dashboard
2	Home page animations: animated gradient headline, drifting color blobs, 3D tilt on the demo card (move your mouse over it), scroll-reveal sections, floating GPS badge	Landing page
3	Checklist count generator: "Number of checklist items" (0–50)	Create-a-job form

#	Change	Where to see it
	above the task list — typing a number instantly creates that many task boxes	
4	Platform fee removed (UI row gone + <code>PLATFORM_FEE_BPS=0</code> in the backend)	Create-a-job summary, wallet after approval
5	Lat/lng inputs replaced by a map picker: search an address, "My location", or tap/drag a pin on the map	Create-a-job form

The 5-minute smoke test (if you're in a hurry)

1. Restart the API once so it picks up `fee=0` + any `.env` changes: `kill -f "node dist/index.js"; cd NetworkPeer-main && npm run build && node dist/index.js`
2. `localhost:8080` → check the big gradient **NetworkPeers** wordmark, animated hero, 3D-tilt demo card (hover it), and that stat cards on `/client` have larger text.
3. Normal window: sign up client, then `/client/jobs/new` : set checklist count to `3` → three task boxes appear; search an address on the map and tap a spot; budget `500` → **Post job**.
4. Click **Fund escrow** on the job → in a terminal: `node scripts/simulate-payment-webhook.mjs <operationId> <providerReference>` → status becomes `POSTED` .
5. Incognito window: sign up worker → accept the job → run the task statuses.
6. Evidence + submit need S3 keys in `.env` (see §3). Approve + wallet shows full payout, no fee.

1. Right now on your Mac (everything is already running)

Two servers must be up. If they are, skip to **Step 3**.

```
# Terminal A – API (restart it so it picks up the new fee=0 + any
  config)
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
kill -f "node dist/index.js" || true
npm run build && node dist/index.js

# Terminal B – frontend (if not running)
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-platform-
  main
npm run dev
```

Check: `curl http://localhost:3000/api/v1/live` →
`{"success":true,...}` and `http://localhost:8080` loads.

One-command option: `./scripts/start-demo.sh` detects what's running and starts the rest. `./scripts/stop-demo.sh` stops everything.

Full feature test — the “boss demo” rehearsal

- 1. Home page** — open `http://localhost:8080`. Watch the animated gradient headline and blobs; hover the demo card (it tilts in 3D); scroll — sections fade up.
- 2. Sign in** — normal window = **client**, incognito window = **worker** (`/auth`). Sign up the worker with role **Worker**. OTP delivery: in console mode the code is shown on the verify screen; with Twilio enabled (§7) a **real SMS** arrives instead — use a phone you can receive on.
- 3. Admin + worker verification** (one time, in a third terminal):

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
docker exec networkpeer-postgis psql -U postgres -d networkpeer -
  f scripts/provision-admin.sql
```

Then verify the worker (replace the phone numbers with the ones you signed up with):

```
docker exec networkpeer-postgis psql -U postgres -d networkpeer \
  --set=admin_phone="+1XXXXXXXXXX" \
  --set=worker_phone="+1XXXXXXXXXX" <<'SQL'
SELECT public.admin_set_worker_verification(
  (SELECT id FROM public.users WHERE phone_number =
    :admin_phone'),
  (SELECT id FROM public.users WHERE phone_number =
    :worker_phone'),
```

```
'VERIFIED', TRUE, 'Verified for live demonstration');
SQL
```

4. Client dashboard — the five stat cards now have larger labels and numbers.

5. Create a job (client window, `/client/jobs/new`): - Type a title + description (10+ chars). - **Checklist**: set "Number of checklist items" to `3` → three task boxes appear instantly. Fill the first task, leave one blank, toggle "Evidence required" on one. Try `0` (list empties) and back up. Try `50` (cap). - **Location**: search "Connaught Place, New Delhi" (or any address) → pin moves; or click "My location"; or tap the map. Watch the coordinates line update. - Budget `500`, leave schedule empty → **Post job**. - It saves as **FUNDING** — invisible to workers until escrow settles.

6. Fund + settle (the money hold): - On the job detail, click **Fund escrow** → copy `operationId` + `providerReference` from the response/UI, then in the backend terminal: `sh cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main node scripts/simulate-payment-webhook.mjs <operationId> <providerReference>` - Refresh → status `POSTED`. (This is what Stripe's webhook would do; the simulator signs the exact same event.)

7. Worker discovers + accepts (incognito window): nearby list shows the job (client identity hidden) → **Accept** → private details reveal. Optional concurrency proof: open a second incognito profile, accept simultaneously — one `200`, one `409`.

8. Live task: Continue live task → EN ROUTE → AT LOCATION → IN PROGRESS.

9. Evidence (S3 — your account is ready, 10 min setup): see section 3 below, then Capture/select evidence → uploads straight to S3 → "Evidence confirmed" → Submit.

10. Client approves → wallet shows the payout with **no platform fee** (fee=0, worker gets the full budget).

11. Realtime: with both windows open, create/accept a job in one and watch the notification bell + live toast in the other.

2. Cloud database (so the data is not tied to this Mac) — recommended before tomorrow

The office laptop can't install anything, and the demo survives Mac restarts better if the database lives in the cloud. The API itself still runs on the Mac (exposed by tunnel); only Postgres moves to the cloud. ~15 minutes, free tier.

1. **Create a free Neon Postgres** (console.neon.tech → New project → region near you). Copy the connection string (it looks like

```
postgresql://user:pass@ep-xxx.region.aws.neon.tech/neondb?
sslmode=require ). PostGIS is available on Neon. (Alternatives: Supabase
free tier, or Railway's managed Postgres.)
```

2. **Migrate + provision roles** (from the Mac, once, with the migration-owner string):

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
export DATABASE_URL="postgresql://user:pass@ep-xxx.../neondb?
sslmode=require"
npm run migrate
psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f scripts/provision-
app-role.sql
```

Generate the four role passwords first with `openssl rand -hex 32 (x4)` and export them as `NETWORKPEER_APP_DB_PASSWORD`, `NETWORKPEER_ADMIN_DB_PASSWORD`, `NETWORKPEER_MEDIA_DB_PASSWORD`, `NETWORKPEER_FINANCIAL_DB_PASSWORD` — the script reads them from the environment.

3. **Point the local API at the cloud DB** — edit `NetworkPeer-main/.env` :

```
DATABASE_URL=postgresql://networkpeer_app:<app-pass>@ep-
xxx.../neondb?sslmode=require
DATABASE_ADMIN_URL=postgresql://networkpeer_admin_api:<admin-
pass>@ep-xxx.../neondb?sslmode=require
DATABASE_MEDIA_VERIFIER_URL=postgresql://networkpeer_media_verif.
<media-pass>@ep-xxx.../neondb?sslmode=require
DATABASE_FINANCIAL_URL=postgresql://networkpeer_financial_api:
<financial-pass>@ep-xxx.../neondb?sslmode=require
```

4. Restart the API (`kill -f "node dist/index.js"; node dist/index.js`)
→ `/health` shows `database: true` . Re-run the demo — same app, cloud-stored data.

Keep the four role passwords somewhere safe. Never commit `.env`.

3. S3 evidence (your AWS account is live — 10 minutes)

1. Create (or reuse) a bucket: **versioning ON, default encryption ON, all public access blocked**.
2. Create an IAM user with programmatic keys; attach a policy granting on that bucket (use these **exact** IAM action names — two older docs listed wrong ones, and `PutBucketCORS` was missing entirely): `s3:PutObject`, `s3:GetObject`, `s3:PutObjectTagging`, `s3:GetBucketVersioning`, `s3:GetBucketPublicAccessBlock`, `s3:GetEncryptionConfiguration`, `s3:PutBucketCORS`. Verify + set CORS with: `node scripts/verify-s3-setup.mjs` (all ) .
3. Add to `NetworkPeer-main/.env` :

```
AWS_REGION=<your-region e.g. ap-south-1>
AWS_ACCESS_KEY_ID=<key>
AWS_SECRET_ACCESS_KEY=<secret>
AWS_S3_BUCKET=<bucket-name>
```

4. Bucket CORS (browser uploads straight from `localhost:8080` ; replace the origin with the tunnel URL for tomorrow):

```
cat > /tmp/networkpeer-s3-cors.json <<EOF
{ "CORSRules": [ { "AllowedHeaders": ["*"], "AllowedMethods":
    ["POST"],
    "AllowedOrigins": ["http://localhost:8080"],
    "ExposeHeaders": ["ETag"], "MaxAgeSeconds": 300 } ] }
EOF
aws s3api put-bucket-cors --bucket "$AWS_S3_BUCKET" --cors-
configuration file:///tmp/networkpeer-s3-cors.json
```

5. Restart the API. Now Step 9 of the demo (evidence upload) works end-to-end — the worker uploads to S3, the API verifies size/type/SHA-256/version, and the DB records it.

3.5 Twilio real-SMS OTP (skipped — needs a paid

Twilio plan)

The Twilio provider is **already implemented** in the backend (fetch-based, no SDK) and was verified to boot + route correctly, but enabling it requires a paid Twilio plan (the trial only sends to verified numbers and the user chose to skip it). The demo therefore runs with `SMS_PROVIDER=console` + `OTP_ECHO_IN_RESPONSE=true` everywhere (local AND cloud demo mode): **the OTP is shown on the verify screen** — any phone number works, no SMS needed.

If you ever want real SMS: set `SMS_PROVIDER=twilio`, `OTP_ECHO_IN_RESPONSE=false`, and `TWILIO_ACCOUNT_SID` / `TWILIO_AUTH_TOKEN` / `TWILIO_FROM_NUMBER` (all 3 required or the API refuses to boot, by design). The frontend already switches to “Verification code sent” when the echo is off.

4. Tomorrow on the Linux office laptop (browser only)

THE MAC WILL NOT BE IN THE OFFICE — the tunnel approach below is OFF the table. `DEPLOY_CLOUD_LINUX_ONLY.md` is now **THE** path: Railway API + Postgres + Redis, Vercel frontend, and a browser funding-settler at `/dev/settle-funding`. Work through it tonight (2–4 h); `LINUX_TOMORROW_QUICKSTART.md` is the condensed action list + shipping checklist.

On the Mac (before the meeting)

```
cd /Users/adityasharma/Desktop/NETWORKPEER/NetworkPeer-main
./scripts/start-demo.sh --public
```

This starts/checks everything and prints two public `https://...` `trycloudflare.com` URLs (API + frontend). Keep the Mac awake and online; **do not restart the tunnels** — the URLs change on every run.

On the Linux laptop

- Open the **frontend URL** in a normal window (client) and an incognito window (worker). Nothing to install; no Node, no Docker — just internet + a browser.
- Corporate network must allow HTTPS outbound (it will).

- If the tunnel is blocked, fall back: share your phone's hotspot on the Mac and use the same URLs, or run `./scripts/start-demo.sh` (without `--public`) and demo from the Mac only.

What to check on the Linux laptop

Same boss-demo flow as section 1, steps 1–11. The app is identical — the browser only renders; the Mac serves the API + database (or the cloud DB + S3, which works from anywhere).

5. Quick reference

```
# Servers
curl http://localhost:3000/api/v1/live           # API
curl http://localhost:3000/api/v1/health        # DB + PostGIS
./scripts/start-demo.sh [--public]              # start everything (+
tunnels)
./scripts/stop-demo.sh                         # stop everything

# Funding without Stripe
node scripts/simulate-payment-webhook.mjs <operationId>
                                     <providerReference>

# Database (local)
docker exec networkpeer-postgis psql -U postgres -d networkpeer \
-c "SELECT filename FROM schema_migrations ORDER BY filename
DESC LIMIT 3;"
```

Known limits to mention honestly in the demo: evidence needs S3 configured (section 3); admin screens are still prototypes; browser push (FCM) is not wired; Twilio real-SMS was skipped (paid plan) so the OTP is echoed on the verify screen in demo mode everywhere.

6. What to send to the office laptop before sleeping

Fresh guide for tomorrow: `docs/LINUX_TOMORROW_QUICKSTART.md` — decision table (tunnel vs cloud), tonight's 15-min Mac prep (incl. the one-time S3 IAM fix), the exact Linux-browser execution steps, the boss-demo script, fallbacks, and the final ship list.

The Linux laptop needs **nothing but a browser** — no code, no installs. Email or Teams-message these three things tonight (or first thing tomorrow):

1. **NetworkPeer-**
`main/docs/EXECUTIVE_PRESENTATION_AND_DEPLOYMENT_GUIDE.docx` — the

demo script with talking points (also the `.md` if email is fine with it).

2. **NetworkPeer-main/docs/NETWORKPEER_COMPLETE_GUIDE.docx** — the full technical breakdown (in case anyone asks hard questions).
3. **The two URLs** (only after the cloud deploy is live, tomorrow morning at the latest):
 - o `https://<vercel-origin>` — the app (open normal + incognito windows)
 - o `https://<vercel-origin>/dev/settle-funding` — the funding settlerIf you're presenting via the tunnel instead (Mac available): the two trycloudflare URLs.

That's the entire handover. The office machine never runs the app — it only opens links. > If you are not presenting from this Mac and it is not allowed in the office, use > `DEPLOY_CLOUD_LINUX_ONLY.md` to stand up the cloud deployment tonight.